Generics: شرح التمارين

هنا نحل **التمارين الأربعة المحلولة** في الملف الأساسي، واحداً واحداً. في كل تمرين نمشي بنفس الترتيب: **ماذا يطلبون؟** ← **كيف نفكر؟** ← **نبني الحل جزءاً جزءاً** ← **الكود كاملاً** ← **نتتبع التنفيذ** ← **أخطاء شائعة** ← **اختبر نفسك**. لا تحفظ الحل، تعلّم طريقة التفكير.

> **قبل أن تبدأ:** اقرأ ملف الشرح النظري أولاً (الصف العام، ومعامل النوع، والدالة العامة).

1 التمرين الأول: الصندوق العام Box

المطلوب: صف عام `Box<T>` يخزّن قيمة واحدة، ثم استعماله مرة مع `String` ومرة مع `Integer` ، وطباعة القيمتين.

أولاً: نحلل المطلوب

السؤال	الجواب	ماذا يعني للكود؟
كم نوعاً يحمل الصف؟	نوع واحد يتغير من استعمال لآخر	معامل واحد: `<T>`
ماذا نخزّن؟	قيمة واحدة	حقل واحد: `T value`
كيف تدخل القيمة؟	من البنّاء، وتتغير لاحقاً	بنّاء `Box(T value)` وتابع `set`
كيف تخرج القيمة؟	بنوعها الحقيقي دون Casting	تابع `T get()`
ما الأنواع في الاستعمال؟	نص (`"Java"`) وعدد (`25`)	`Box<String>` و `Box<Integer>`

> **لاحظ:** هذا التمرين هو **نفسه** الصندوق الذي شرحناه في الملف النظري. الفكرة: نأخذ صندوقاً عادياً ونبدّل نوعه بـ `T` .

ثانياً: نبني الحل جزءاً جزءاً

الجزء 1: الهيكل والحقل.

الـكـود

```
public class Box<T> {
    private T value;
}
```

فراغ اسمه T بعد اسم الصف

الحقل من النوع T

الجزء 2: البنّاء يستقبل قيمة من النوع `T` .

الـكـود

```
    public Box(T value) {
        this.value = value;
    }
```

الوسيط من النوع T

نخزّن القيمة

الجزء 3: `get` و `set`، وكلاهما يتعامل مع `T`.

الـكـود

```
    public T get() { return value; }
    public void set(T value) { this.value = value; }
```

ترجع T: لا نحتاج Casting

تقبل T فقط

ثالثاً: الكود كاملاً والاستعمال

Box.java

```
package generics;

public class Box<T> {
    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }

    public void set(T value) {
        this.value = value;
    }
}
```

الصف العام

حقل نوعه T

البنّاء

ترجع T

تقبل T

الاستعمال (في main)

```
Box<String> b1 = new Box<>("Java");
Box<Integer> b2 = new Box<>(25);
String s = b1.get();
int n = b2.get();
System.out.println(s + " " + n);
```

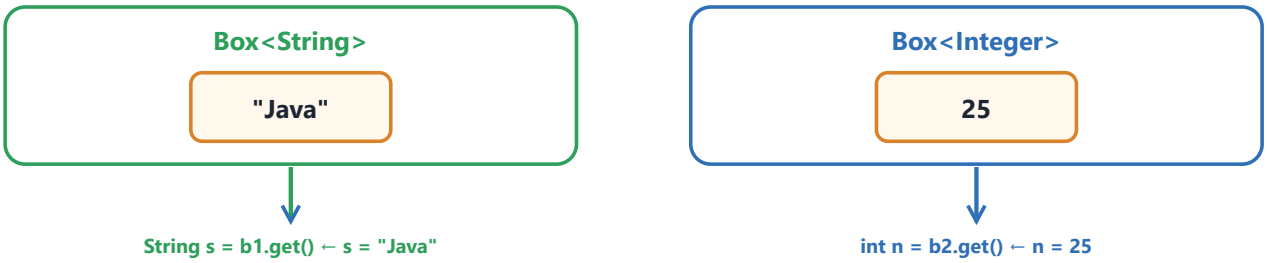
هنا T أصبحت String

هنا T أصبحت Integer: 25 تتحول تلقائياً (Autoboxing)

ترجع String مباشرة

ترجع Integer ويحولها إلى int (Unboxing)

Box<String> b1 = new Box<>("Java"); Box<Integer> b2 = new Box<>(25);



الناتج الفعلي

```
Java 25
```

رابعاً: أخطاء شائعة

وضعت قيمة من نوع مخالف

`b.set(5);` وهو `Box<String>` :

الخطأ الفعلي من المترجم:

```
incompatible types: int cannot be converted to String
```

لأن `T` هنا `String` فقط. وهذا بالضبط ما يطلبه التمرين في آخره: "جرّب تمرير قيمة من نوع مخالف واكتب في تعليق ما يحدث". **الجواب:** خطأ ترجمة، ولا يعمل البرنامج.

قرأت القيمة في متغير من نوع آخر

`Integer n = b.get();` وهو `Box<String>` :

الخطأ الفعلي من المترجم:

```
incompatible types: String cannot be converted to Integer
```

لأن `get()` ترجع `String` . المترجم يعرف النوع، فلا يسمح لك بخلطه.

خامساً: اختبر نفسك

❓ **فكّر: 1) ما نوع ما ترجعه `b2.get()` ؟**
الجواب: `Integer` ، وجافا تحوّله إلى `int` تلقائياً (Unboxing).

❓ **فكّر: 2) لماذا كتبنا `Box<Integer>` وليس `Box<int>` ؟**
الجواب: لأن معامل النوع يجب أن يكون **صفاً**، و `int` نوع أولي. نستعمل المغلّف `Integer` .

❓ **فكّر: 3) ماذا يعني `<>` في `new Box<>(25)` ؟**
الجواب: Diamond: المترجم يستنتج النوع من الطرف الأيسر (`Integer`).

2 التمرين الثاني: الزوج Pair وتبديل القيمتين

المطلوب: صف عام `Pair<K, V>` يحفظ مفتاحاً وقيمة (من نوعين قد يختلفان)، وفيه دالة `swap` تعكس المفتاح والقيمة، ودالة `toString` للطباعة. ثم جرّبه بـ `("Ali", 20)` .

أولاً: نحلل المطلوب

السؤال	الجواب	ماذا يعني للكود؟
كم نوعاً يحمل الصف؟	نوعان: المفتاح والقيمة، وقد يختلفان	معاملان: `<K, V>`
ماذا نخزّن؟	مفتاح وقيمة	حقلان: `K key` و `V value`
ماذا يفعل `swap` ؟	يرجع زوجاً جديداً مقلوب الترتيب	**ينشئ** زوجاً جديداً ولا يغيّر القديم
ما نوع الزوج الجديد؟	القيمة أولاً ثم المفتاح، فالنوعان انقلبا أيضاً	النوع هو `Pair<V, K>` (ليس `Pair<K, V>` !)
كيف نطبعه؟	بشكل مقروء مثل (Ali, 20)	`toString()`

أهم نقطة في التمرين: الدالة `swap` ترجع `Pair<V, K>` وليس `Pair<K, V>` . لأن الذي كان **مفتاحاً** من نوع `K` صار **قيمة**، والعكس. فنوعا الزوج الجديد معكوسان.

ثانياً: نبني الحل جزءاً جزءاً

الجزء 1: الهيكل والحقلان.

الـكـود

```
class Pair<K, V> {
    private K key;
    private V value;
}
```

معاملا نوع: K و V

حقل من نوع K

حقل من نوع V

الجزء 2: البنّاء يستقبل قيمتين بنوعيهما ويخزنهما.

الـكـود

```
    Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }
```

الوسيط الأول K والثاني V

نخزّن بالترتيب نفسه

الجزء 3: الدالة `swap` : هنا نتوقف ونفكر في **نوع الإرجاع**.

الـكـود

```
    Pair<V, K> swap() {
        return new Pair<>(value, key);
    }
```

نوع الإرجاع: V ثم K (معكوس)

نمرر القيمة أولاً ثم المفتاح

لماذا `Pair<V, K>` وليس `Pair<K, V>` ؟

نسأل: ما نوع **الوسيط الأول** في `new Pair<>(value, key)` ؟ هو `value` من نوع `V` . إذن في الزوج الجديد **الأول** (`K` الجديدة) من نوع `V` ، و**الثاني** من نوع `K` . فالنوعان انعكسا: `Pair<V, K>` .

ولو كتبنا `Pair<K, V>` لوعدنا المترجم بشيء غير صحيح، فيرفض. وهذا مثال على أن Generics تحرس أنواعك.

الجزء 4: `toString` للطباعة.

الـكـود

```
    @Override
    public String toString() {
        return "(" + key + ", " + value + ")";
    }
```

نعيد كتابة toString الموجود في Object

نبني نصاً مثل (Ali, 20)

ثالثاً: الكود كاملاً

Pair.java

```java
package generics;

public class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public Pair<V, K> swap() {
        return new Pair<>(value, key);
    }

    @Override
    public String toString() {
        return "(" + key + ", " + value + ")";
    }
}
```

الخطوة 1: معاملان

حقل بنوع K

حقل بنوع V

البنّاء يخزن القيمتين

زوج جديد بأنواع معكوسة

ننشئ زوجاً جديداً: القيمة أولاً

للطباعة المقروءة

الاستعمال

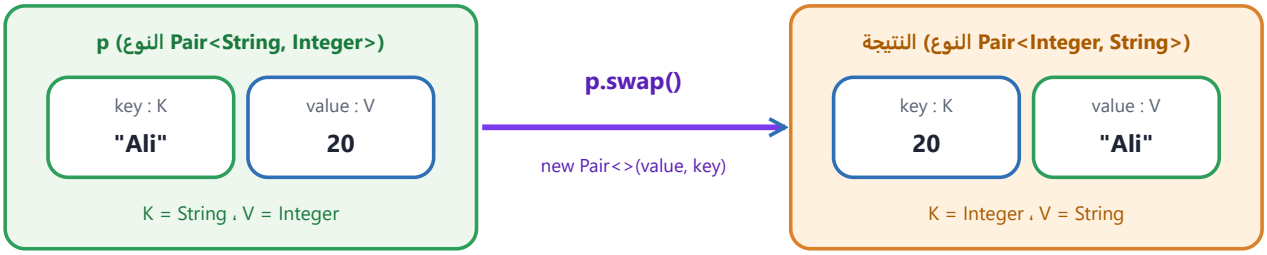
```java
Pair<String, Integer> p = new Pair<>("Ali", 20);
System.out.println(p);
System.out.println(p.swap());
```

V = Integer و K = String

تستدعي toString تلقائياً

زوج جديد: Pair

Pair<String, Integer> p = new Pair<>("Ali", 20);



القيمتان تبدّلتا مكانيهما، وأنواعهما تبدّلت أيضاً: Pair<V,K> صار Pair<K,V>

الناتج الفعلي

```
(Ali, 20)
(20, Ali)
```

لاحظ أن `p` نفسه **لم يتغير**: `swap()` أنشأ زوجاً جديداً وأرجعه، فبقي الأصل `(Ali, 20)` .

رابعاً: أخطاء شائعة

نسيت أن النوع ينعكس، فكتبت `(key, value)` في `swap`

لو كتبنا `return new Pair<>(key, value);` وكان نوع الإرجاع `Pair<V, K>` يعترض المترجم:

الخطأ الفعلي من المترجم:

```
incompatible types: cannot infer type arguments for Pair<>
reason: inference variable K#1 has incompatible bounds
```

لأن الوسيط الأول صار من نوع `K` لكن المطلوب `V`. **الإصلاح:** مرّر `value` أولاً ثم `key`.

كتبت المعاملين بترتيب مختلف عند الإنشاء

`Pair<Integer, String> p = new Pair<>("Ali", 20);` خطأ، لأن `K` هي `Integer` فيجب أن يكون الوسيط الأول عدداً. **القاعدة:** ترتيب الأنواع بين `< >` هو نفس ترتيب الوسائط.

خامساً: اختبر نفسك

❓ **فكّر: 1) ما نوع `p.swap()` إذا كان `p` من `Pair<String, Integer>` ؟**

الجواب: `Pair<Integer, String>`.

❓ **فكّر: 2) هل `swap` تغيّر الزوج الأصلي `p` ؟**

الجواب: لا، تنشئ زوجاً جديداً. الأصل يبقى كما هو.

❓ **فكّر: 3) لماذا نحتاج معاملين `K` و `V` وليس معاملاً واحداً `T` ؟**

الجواب: لأن المفتاح والقيمة قد يكونان **نوعين مختلفين** (نص وعدد). مع `T` واحد سيكونان من النوع نفسه.

3 التمرين الثالث: دالتان عامتان للمصفوفات

المطلوب: دالتان عامتان: الأولى `printArray` تطبع عناصر **أي** مصفوفة، والثانية `swap` تبدّل عنصرين فيها.

جرّبهما مع مصفوفة أعداد ومصفوفة نصوص.

أولاً: نحلل المطلوب

السؤال	الجواب	ماذا يعني للكود؟
هل نحتاج صفاً عاماً؟	لا، نحتاج **دالتين فقط** تعملان مع أي نوع	دالتان عامتان وليس صفاً عاماً
ماذا تستقبل `printArray` ؟	مصفوفة من أي نوع	`T[] arr`
ماذا ترجع؟	لا شيء، تطبع فقط	`void`
ماذا تستقبل `swap` ؟	المصفوفة ورقما العنصرين	`T[] arr, int i, int j`
أين نكتب `<T>` ؟	قبل نوع الإرجاع مباشرة	`static <T> void ...`
هل نحتاج متغيراً مؤقتاً؟	نعم، لتبديل عنصرين لا نفقد أحدهما	`T temp` : من نوع `T` أيضاً

ثانياً: نبني الحل جزءاً جزءاً

الجزء 1: `printArray` . نبدأ بالسؤال: ما الذي يتغير بين نوع وآخر؟ **نوع عناصر المصفوفة فقط**، فنجعله `T` .

الـكـود

```
static <T> void printArray(T[] arr) {
    for (T item : arr) {
        System.out.print(item + " ");
    }
    System.out.println();
}
```

قبل void مباشرة

مصفوفة من أي نوع

كل عنصر من نوع T

الجزء 2: `swap` . ما الذي يحتاج نوعاً هنا؟ المصفوفة **و** المتغير المؤقت.

الـكـود

```
static <T> void swap(T[] arr, int i, int j) {
    T temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

معامل نوع خاص بهذه الدالة

المؤقت من النوع T: لهذا نحتاج T داخل الجسم

نضع الثاني مكان الأول

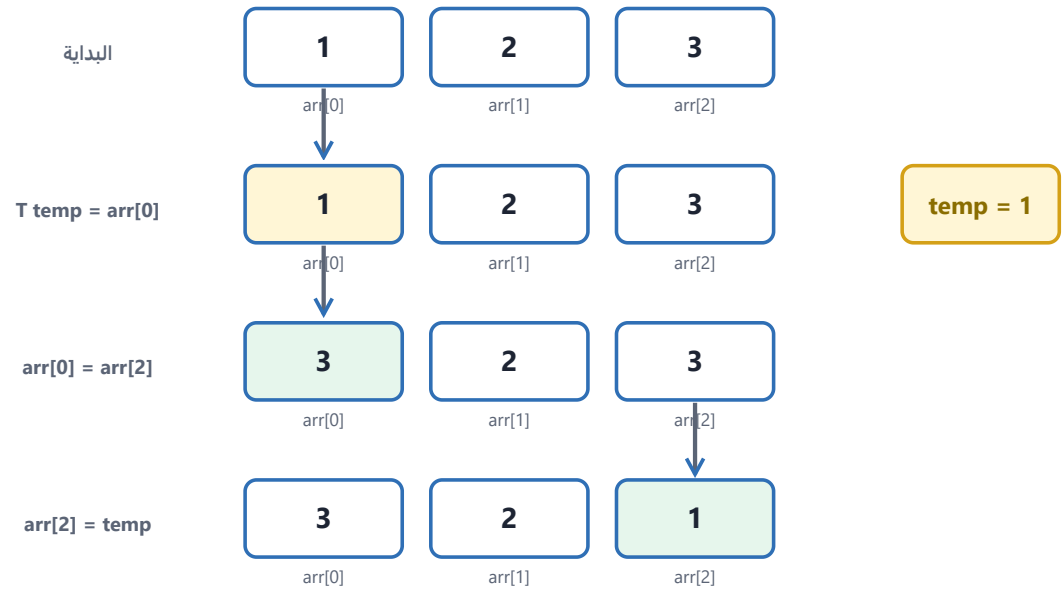
نضع القديم مكان الثاني

لماذا نحتاج `T temp` ؟ هل لا نكتب `arr[i] = arr[j]` فقط؟

لو كتبنا `arr[i] = arr[j];` أولاً **نفقد** القيمة القديمة في `arr[i]` للأبد! لذلك نحفظها أولاً في `temp` . مثل أن تبدّل كأسين: تحتاج كأساً ثالثة فارغة.

ثالثاً: تتبّع التنفيذ

نتتبع `swap(nums, 0, 2)` على المصفوفة `{1, 2, 3}` سطراً سطراً:



رابعاً: الكود كاملاً والاستدعاء

GenericMethods.java

```java
package generics;

public class GenericMethods {

    static <T> void printArray(T[] arr) {
        for (T item : arr) {
            System.out.print(item + " ");
        }
        System.out.println();
    }

    static <T> void swap(T[] arr, int i, int j) {
        T temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    public static void main(String[] args) {
        Integer[] nums = {1, 2, 3};
        String[] names = {"Sara", "Omar"};

        printArray(nums);
        swap(nums, 0, 2);
        printArray(nums);

        printArray(names);
        swap(names, 0, 1);
        printArray(names);
    }
}
```

دالة عامة بمعاملها الخاص

دالة عامة ثانية بمعاملها الخاص

متغير مؤقت من النوع T

Integer[] وليس int[] (الأنواع الأولية لا تصلح)

T = Integer (يُستنتج)

T = String (يُستنتج)

الناتج الفعلي

```
1 2 3
3 2 1
Sara Omar
Omar Sara
```

لاحظ: الدالتان **نفسهما** عملتا مع `Integer[]` ومع `String[]` دون أن نغيّر حرفاً. وهذا هو معنى الدالة العامة. ولم نكتب النوع عند الاستدعاء: المترجم استنتجه.

خامساً: أخطاء شائعة

نسيت كتابة `<T>` قبل نوع الإرجاع

`static void printArray(T[] arr)` بلا `<T>`:

الخطأ الفعلي من المترجم:

```
cannot find symbol
symbol:   class T
location: class E5
```

لأن المترجم لا يعرف ما هو `T` (ليس صفاً موجوداً). **الإصلاح:** اكتب `<T>` قبل `void` لتعرّفه.

استعملت مصفوفة أعداد أولية `int[]`

`int[] c = {1, 2}; printArray(c);`

الخطأ الفعلي من المترجم:

```
method printArray in class GenericMethods cannot be applied to given types;
required: T[]
found: int[]
reason: inference variable T has incompatible bounds
```

لأن `T` لا يكون نوعاً أولياً. **الإصلاح:** استعمل `Integer[]` بدل `int[]`.

سادساً: اختبر نفسك

❓ **فكّر: 1) أين يُكتب `<T>` في الدالة العامة؟**

الجواب: قبل نوع الإرجاع مباشرة: `static <T> void ...`.

❓ **فكّر: 2) متى يتحدد `T` في الدالة العامة؟**

الجواب: عند **كل استدعاء**، والمترجم يستنتجه من الوسيط (Type Inference).

❓ **فكّر: 3) هل `printArray(nums)` و `printArray(names)` يستعملان `T` نفسه؟**

الجواب: لا. في الأول `T = Integer` وفي الثاني `T = String`. الدالة واحدة لكن `T` يتحدد في كل استدعاء.

❓ **فكّر: 4) لماذا يجب أن يكون `temp` من النوع `T` ؟**

الجواب: لأنه يحمل عنصراً من المصفوفة، وعناصرها من النوع `T`.

4 التمرين الرابع: واجهة عامة Converter

المطلوب: واجهة عامة `Converter<A, B>` وصف `LengthConverter` ينفذها بأنواع محددة (يحوّل نصاً إلى طوله)، ثم تجربتها في `main`.

أولاً: نحلل المطلوب

السؤال	الجواب	ماذا يعني للكود؟
ماذا تعد الواجهة؟	تابع يحوّل قيمة من نوع إلى نوع آخر	تابع واحد: `B convert(A input)`
كم نوعاً مفتوحاً؟	نوع المدخل ونوع المخرج	معاملان: `<A, B>`
من يقرر النوعين؟	الصف الذي **ينفّذ** الواجهة	`implements Converter<String, Integer>`
ماذا يفعل `LengthConverter`؟	يأخذ نصاً ويرجع طوله	`return s.length()`

ثانياً: نبني الحل جزءاً جزءاً

الجزء 1: الواجهة: نعد بتابع ولا نكتب له جسماً.

الـكـود

```
interface Converter<A, B> {
    B convert(A input);
}
```

فراغان: A للمدخل و B للمخرج

تستقبل A وترجع B، بلا جسم

الجزء 2: الصف: نقرر ما هما A و B، فتصبح الواجهة محددة.

الـكـود

```
class LengthConverter implements Converter<String, Integer> {
    public Integer convert(String s) {
        return s.length();
    }
}
```

قررنا: A = String و B = Integer

التابع يطابق الأنواع: يستقبل String ويرجع Integer

نرجع طول النص

كيف تنظر الواجهة إلى الصف بعد أن حددنا الأنواع؟

عندما كتبنا `Converter<String, Integer>` استبدل المترجم `A` بـ `String` و `B` بـ `Integer` في الواجهة، فصارت كأنها:

```
interface Converter {
    Integer convert(String input);
}
```

ولذلك **يجب** أن يكتب الصف التابع بهذه الأنواع بالضبط (يستقبل `String` ويرجع `Integer`)، وإلا يعترض المترجم.

ثالثاً: الكود كاملاً والاستعمال

Converter.java

```
package generics;

public interface Converter<A, B> {
    B convert(A input);
}
```

واجهة عامة بمعاملين

تابع بلا جسم

LengthConverter.java

```
package generics;

public class LengthConverter implements Converter<String, Integer> {
```

حددنا الأنواع

```
    @Override
    public Integer convert(String s) {
        return s.length();
    }
}
```

يطابق الأنواع المحددة

الطول

الاستعمال (في main)

```
Converter<String, Integer> c = new LengthConverter();
System.out.println(c.convert("Generics"));
```

المتغير من نوع الواجهة بالأنواع نفسها

والغرض من الصف الذي ينفّذها

ندخل نصاً ويخرج عدد بلا Casting

الناتج الفعلي

```
8
```

لماذا 8؟ لأن كلمة `Generics` فيها 8 أحرف: G-e-n-e-r-i-c-s.

رابعاً: أخطاء شائعة

كتبت نوع الإرجاع مخالفاً للأنواع المحددة

`public String convert(String s)` داخل صف يقول `Converter<String, Integer>` :

الخطأ الفعلي من المترجم:

```
convert(String) in E11 cannot implement convert(A) in Converter
return type String is not compatible with Integer
```

لأن الواجهة وعدت بـ `Integer` لا `String` . **الإصلاح:** اجعل نوع الإرجاع `Integer` .

مرّرت عدداً بدل النص

`c.convert(5);`

الخطأ الفعلي من المترجم:

```
incompatible types: int cannot be converted to String
```

لأن `A` هي `String` . وهذا ما يطلبه الملف الأساسي من تجربة "محول لا يناسب النوع": الجواب خطأ ترجمة.

خامساً: اختبر نفسك

❓ **فكّر: 1) ما معنى `A` و `B` في الواجهة؟**

الجواب: `A` نوع المدخل و `B` نوع المخرج. نتركهما مفتوحين ويقرّرهما الصف الذي ينفّذ الواجهة.

❓ **فكّر: 2) هل يمكن أن يبقى الصف عاماً وينفّذ الواجهة؟**

الجواب: نعم: `class X<A, B> implements Converter<A, B>` يمرّر المعاملين دون أن يحددهما.

❓ **فكّر: 3) لماذا لا نكتب جسماً للتابع داخل الواجهة؟**

الجواب: الواجهة **وعد** فقط. الصف الذي ينفّذها هو الذي يكتب الكود بطريقته.

5 كيف أبدأ أي تمرين Generics جديد؟

#	اسأل نفسك	ماذا تكتب؟
1	هل أحتاج **صفاً** يحمل نوعاً، أم **دالة** فقط تعمل مع أي نوع؟	صف عام `<T>` بعد اسم الصف، أو دالة عامة `<T>` قبل نوع الإرجاع
2	كم نوعاً **مختلفاً** سيحمله؟	نوع واحد `T` ، أو عدة `<K, V>`
3	ما الذي **يتغير** بين نوع وآخر؟	نكتب مكانه `T` ، وما لا يتغير يبقى كما هو
4	هل نوع الإرجاع يعتمد على الأنواع المعطاة؟	اكتبه بالمعاملات (مثل `Pair<V, K>`) بدل نوع ثابت
5	هل سأضع أعداداً أولية؟	استعمل الصف المغلّف: `Integer` و `Double`

تذكّر: نكتب الكود **مرة واحدة** بفراغ اسمه `T` ، وعند الاستعمال نملأ الفراغ، والمترجم يحرس أن لا يدخل نوع غير مناسب.